

VetCare System

Sistema de Gestión Clínica Veterinaria

Proyecto de Aula

Integrantes

Sara Corrales Jaramillo | Manuela Escobar Hernández

Profesor

Joseph Mateus Raigoza Mesa

Arquitectura de Software · Medellín, 2026

1. Descripción del Problema

En Colombia, aproximadamente entre el 60 % y el 67 % de los hogares cuentan con al menos una mascota (DANE; Fenalco). Este crecimiento sostenido ha incrementado la demanda de servicios veterinarios y la necesidad de gestionar información clínica de forma eficiente y centralizada.

La situación actual presenta cifras alarmantes que justifican la necesidad del sistema:

- ~63 % de hogares colombianos poseen al menos una mascota (DANE, 2024).
- ~40 % de las mascotas no cumplen esquemas completos de vacunación ni controles periódicos.
- Entre el 30 % y el 50 % de los diagnósticos veterinarios se realizan de forma tardía.
- Una gran proporción de clínicas opera con sistemas aislados o registros no estandarizados, dificultando la interoperabilidad.

Se requiere un sistema centralizado — VetCare System — que permita recolectar, almacenar, procesar y consultar información clínica de múltiples servicios de forma segura, escalable y observable, soportando monitoreo proactivo, diagnóstico rápido de incidencias y análisis en tiempo real.

2. Abstract

Keywords: *microservices, veterinary clinic management, software architecture, REST API, OpenAPI, quality attributes, scalability, security.*

In Colombia, approximately 63% of households own at least one pet, generating a growing demand for efficient and centralized veterinary clinical management systems. Current solutions rely on isolated records and non-standardized tools, resulting in approximately 40% of pets failing to complete vaccination schedules and 30–50% of diagnoses being made too late.

VetCare System addresses this gap by proposing a microservices-based architecture composed of six independent services: user management, clinical history, tracking and reminders, storage, query visualization, and notifications. Each service is independently deployable and communicates through well-defined REST APIs documented with OpenAPI 3.0 specifications, enabling interoperability, contract clarity, and continuous integration.

The architecture prioritizes four key quality attributes: security (role-based access control per service, access denied in under 2 seconds), reliability (failover with recovery under 5 seconds, error rate below 1%), performance efficiency (response times under 2 seconds for up to 1,000 concurrent users), and scalability (horizontal scaling maintaining CPU below 80% during traffic peaks). Discarded alternatives — monolithic, N-tier, SOA, and serverless — were rejected due to limitations in independent scalability, single points of failure, cold-start latency, and deployment constraints.

The repository adopts a monorepo structure with explicit separation per microservice under a /services/ directory, complemented by shared infrastructure and documentation directories. All architectural decisions are captured in three Architecture Decision Records (ADRs): selection of microservices style, internal repository structure, and REST communication strategy with OpenAPI contracts.

3. Atributos de Calidad

El sistema VetCare está diseñado en torno a cuatro atributos de calidad principales, cada uno respaldado por escenarios concretos y medibles:

3.1 Seguridad

Justificación: El sistema maneja información clínica sensible de mascotas y datos personales de sus dueños. Es fundamental garantizar que solo los usuarios autorizados puedan acceder o modificar registros, cumpliendo con principios de mínimo privilegio y control de acceso por roles (RBAC).

Escenario 1: Acceso controlado a información clínica	
Fuente del estímulo	Usuario no autorizado
Estímulo	Intento de acceso a historial clínico sin permisos
Artefacto	Servicio de gestión de usuarios (user-management)
Entorno	Operación normal
Respuesta	El sistema bloquea el acceso y solicita autenticación válida
Medida	Acceso denegado en < 2 s

Escenario 2: Protección de modificación de datos clínicos	
Fuente del estímulo	Usuario autenticado sin permisos suficientes
Estímulo	Intento de edición de historial clínico
Artefacto	Servicio de historial clínico (clinical-history)
Entorno	Operación normal
Respuesta	El sistema bloquea la acción y notifica falta de permisos
Medida	Bloqueo inmediato en < 1 s

3.2 Fiabilidad

Justificación: Un sistema de salud veterinaria debe garantizar la disponibilidad continua del historial clínico. La pérdida o corrupción de datos puede comprometer la salud de las mascotas. El aislamiento de fallos entre microservicios evita que una caída afecte a todo el sistema.

Escenario 3: Disponibilidad del sistema ante fallo	
Fuente del estímulo	Falla en el servidor principal
Estímulo	Caída del servicio de almacenamiento
Artefacto	Servicio de almacenamiento (storage-service)
Entorno	Alta demanda
Respuesta	El sistema activa el nodo de respaldo y continúa la operación
Medida	Tiempo de recuperación < 5 s

Escenario 4: Registro correcto de información clínica	
Fuente del estímulo	Veterinario
Estímulo	Registro de una nueva consulta médica
Artefacto	Servicio de historial clínico (clinical-history)
Entorno	Operación normal
Respuesta	El sistema guarda la información correctamente y confirma al usuario
Medida	Tasa de error < 1 %

3.3 Eficiencia de Desempeño

Justificación: Los veterinarios necesitan acceder rápidamente al historial clínico durante las consultas. Demoras afectan la experiencia del usuario y la calidad de atención. Con hasta 1.000 usuarios concurrentes en horas pico, el sistema debe mantener tiempos de respuesta aceptables.

Escenario 5: Consulta rápida del historial clínico	
Fuente del estímulo	Usuario (veterinario o dueño)
Estímulo	Solicitud del historial clínico de una mascota
Artefacto	Servicio de consulta y visualización (query-visualization)
Entorno	1.000 usuarios concurrentes

Escenario 5: Consulta rápida del historial clínico	
Respuesta	El sistema muestra la información solicitada
Medida	Tiempo de respuesta < 2 s

Escenario 6: Soporte a múltiples usuarios simultáneos	
Fuente del estímulo	Usuarios concurrentes
Estímulo	Incremento repentino de solicitudes (pico de tráfico)
Artefacto	Servidor de aplicaciones
Entorno	Pico de tráfico
Respuesta	El sistema mantiene el rendimiento mediante escalado horizontal
Medida	CPU < 80 % y tiempo de respuesta < 3 s

3.4 Escalabilidad

Justificación: La demanda de servicios veterinarios en Colombia ha crecido sostenidamente. El sistema debe poder incorporar nuevos servicios, más usuarios y mayor volumen de datos sin rediseñar la arquitectura existente. La independencia de despliegue de cada microservicio habilita el escalado granular.

Escenario 7: Incorporación de nuevo microservicio	
Fuente del estímulo	Equipo de desarrollo
Estímulo	Necesidad de agregar módulo de telemedicina veterinaria
Artefacto	Repositorio monorepo (/services/)
Entorno	Sistema en producción
Respuesta	El nuevo servicio se integra sin modificar los existentes ni interrumpir la operación
Medida	Tiempo de integración < 1 sprint (2 semanas), cero downtime en servicios existentes

Escenario 8: Escalado horizontal bajo carga sostenida	
Fuente del estímulo	Plataforma de orquestación (Kubernetes)
Estímulo	Incremento sostenido del 300% en solicitudes al servicio de historial clínico

Escenario 8: Escalado horizontal bajo carga sostenida	
Artefacto	Servicio clinical-history + infraestructura Kubernetes
Entorno	Operación normal con crecimiento de usuarios
Respuesta	Kubernetes despliega réplicas adicionales del servicio afectado sin escalar los demás
Medida	Latencia < 2 s mantenida; recursos de otros servicios sin variación

ADR 001: Selección de Estilo Arquitectónico

Sistema de Gestión Clínica Veterinaria — VetCare System

Información General

Campo	Valor
ID	ADR-001
Título	Selección de Estilo Arquitectónico
Estado	Aprobado
Fecha	2026
Autoras	Manuela Escobar Hernández Sara Corrales Jaramillo
Proyecto	Sistema de Gestión Clínica Veterinaria
Docente	Joseph Mateus Raigoza Mesa

1. Contexto

En Colombia, aproximadamente entre el 60 % y el 67 % de los hogares cuentan con al menos una mascota (DANE; Fenalco). Este crecimiento ha incrementado la demanda de servicios veterinarios y la necesidad de gestionar información clínica de forma eficiente.

Actualmente, una gran proporción de clínicas veterinarias opera con sistemas aislados o registros no estandarizados, lo que dificulta la interoperabilidad de la información. Cerca del 40 % de las mascotas no cumplen esquemas completos de vacunación ni controles periódicos, y entre el 30 % y el 50 % de los diagnósticos veterinarios se realizan de forma tardía.

Se requiere un sistema centralizado que permita recolectar, almacenar, procesar y consultar información clínica de múltiples servicios de forma segura, escalable y observable, soportando monitoreo proactivo, diagnóstico rápido de incidencias y análisis en tiempo real.

2. Decisión

Se adopta el estilo arquitectónico de **Microservicios** para el sistema de gestión clínica veterinaria VetCare System.

El sistema estará compuesto por seis microservicios independientes, cada uno con responsabilidades claramente delimitadas, despliegue autónomo y comunicación a través de APIs bien definidas.

3. Microservicios del Sistema

Microservicio	Responsabilidad
user-management	Administración de usuarios (veterinarios y dueños), autenticación, autorización y control de acceso por roles (RBAC).
clinical-history	Registro, almacenamiento y consulta de información médica: consultas, diagnósticos, tratamientos y vacunas.
tracking-reminders	Gestión de alertas preventivas: vacunas pendientes, controles periódicos y seguimiento de tratamientos.
storage-service	Base de datos centralizada con mecanismos de respaldo y recuperación, garantizando persistencia y disponibilidad.
query-visualization	Interfaces para que veterinarios y dueños accedan al historial clínico con visualización clara de datos.
notification-service	Envío de alertas y recordatorios multicanal (app móvil, correo electrónico, notificaciones push).

4. Alternativas Descartadas

Alternativa	Razón del descarte
Arquitectura Monolítica	Un único bloque de código dificultaría la escalabilidad independiente de componentes. Un fallo en un módulo podría comprometer todo el sistema y los despliegues requerirían detener la aplicación completa.
Arquitectura en Capas (N-Tier)	No resuelve la escalabilidad granular. Los servicios de alta demanda no podrían escalarse sin escalar toda la capa de negocio, encareciendo la infraestructura.
Arquitectura Orientada a Servicios (SOA)	Depende de un Enterprise Service Bus (ESB) centralizado que introduce un punto único de fallo y mayor latencia. Desproporcionado para el alcance del proyecto.
Arquitectura Serverless	El 'cold start' puede incumplir los SLAs de tiempo de respuesta (< 2 s). Dificulta la gestión de estado en flujos de historial clínico y el control sobre datos sensibles.

5. Razones de la Decisión

- **Escalabilidad independiente:** cada microservicio puede escalar según su propia carga sin afectar a los demás.
- **Tolerancia a fallos:** el aislamiento garantiza que un fallo en notificaciones no comprometa el historial clínico ni la autenticación.
- **Seguridad granular:** el control de acceso por roles (RBAC) se implementa a nivel de cada servicio.

- **Integración continua:** los microservicios pueden desplegarse, actualizarse y revertirse de forma independiente.
- **Observabilidad:** la arquitectura facilita instrumentar cada servicio con métricas, trazas distribuidas y logs estructurados.
- **Adaptabilidad:** nuevos servicios pueden incorporarse sin modificar los existentes.

6. Consecuencias

Positivas

- **Alta disponibilidad:** los fallos quedan contenidos en el microservicio afectado.
- **Despliegue continuo:** pipelines CI/CD independientes por servicio permiten entregas frecuentes y de bajo riesgo.
- **Tecnología heterogénea:** cada servicio puede usar el lenguaje o base de datos más adecuado para su función.
- **Equipos autónomos:** trabajo en paralelo sobre servicios distintos sin conflictos de integración constantes.

Riesgos y Mitigación

Riesgo	Mitigación
Mayor complejidad operacional	Implementar monitoreo (Prometheus, Grafana), trazabilidad distribuida (Jaeger) y un API Gateway centralizado.
Latencia entre servicios	REST con timeouts estrictos en todas las llamadas entre servicios. El diseño de servicios cohesivos minimiza las cadenas de llamadas síncronas (máximo 2 saltos por operación).
Consistencia eventual de datos	Aplicar el patrón Saga para transacciones distribuidas y contratos de API versionados.
Overhead de infraestructura	Contenerizar con Docker y orquestar con Kubernetes para automatizar despliegue, escalado y recuperación.

7. Definición del Nicho

Usuarios del sistema

- **Usuarios primarios — Veterinarios:** definen los datos obligatorios del sistema (historial clínico, vacunación, diagnósticos, tratamientos).
- **Usuarios secundarios — Dueños de mascotas:** acceden al historial clínico para consulta y seguimiento preventivo. Pueden complementar información opcional: hábitos, alimentación y síntomas.

8. Referencias

- Fowler, M. (2014). Microservices. <https://martinfowler.com/articles/microservices.html>

- Newman, S. (2015). Building Microservices. O'Reilly Media.
- Amazon Web Services. What are Microservices?
<https://aws.amazon.com/microservices/>
- Microsoft (2023). Microservices Architecture Guide. <https://learn.microsoft.com/en-us/dotnet/architecture/microservices/>
- DANE (2024). Encuesta de calidad de vida. <https://www.dane.gov.co>
- Ministerio de Salud y Protección Social (2024). Lineamientos de salud pública y bienestar animal.

ADR 002: Estructura Interna del Repositorio

Sistema de Gestión Clínica Veterinaria — Arquitectura de Microservicios

Información General

Campo	Valor
ID	ADR-002
Título	Estructura Interna del Repositorio
Estado	Aprobado
Fecha	2026
Autoras	Manuela Escobar Hernández Sara Corrales Jaramillo
Proyecto	Sistema de Gestión Clínica Veterinaria
Docente	Joseph Mateus Raigoza Mesa

1. Contexto

El sistema propuesto es una plataforma centralizada de gestión clínica veterinaria basada en una arquitectura de Microservicios. El repositorio del proyecto debe organizarse de forma que refleje claramente esta arquitectura, permita la colaboración eficiente del equipo y facilite el ciclo de integración/despliegue continuo (CI/CD).

Dado que el sistema está compuesto por múltiples servicios independientes, la estructura del repositorio debe ser coherente con esta separación de responsabilidades y facilitar la autonomía de cada microservicio.

2. Decisión

Se adopta una estructura de repositorio tipo **Monorepo** con separación explícita por microservicio, utilizando un directorio raíz `/services/` que contiene cada servicio como módulo independiente, complementado con directorios compartidos para infraestructura, documentación y configuración global.

3. Estructura Propuesta del Repositorio

```
vetcare-system/ |─ services/ |   |─ user-management/           # Gestión de usuarios y
autenticación |   |─ clinical-history/           # Historial clínico de mascotas |   |─
tracking-reminders/           # Seguimiento y recordatorios |   |─ storage-service/           #
Almacenamiento y respaldo de datos |   |─ query-visualization/       # Consulta y visualización
|   |─ notification-service/       # Notificaciones multicanal |─ infrastructure/           #
Docker, Kubernetes, CI/CD |─ shared/           # Utilidades y contratos compartidos
|─ docs/           # ADRs y documentación de arquitectura |   |─ adr-001-
```

4. Alternativas Descartadas

Alternativa	Razón del descarte
Polyrepo (múltiples repos)	Dificulta la gestión de dependencias entre servicios, aumenta la complejidad de coordinación de versiones y fragmenta la visibilidad del proyecto. El overhead supera los beneficios para un equipo pequeño.
Monorepo plano (sin /services/)	Mezcla archivos de distintos microservicios en el mismo nivel, dificultando la comprensión de la arquitectura y la automatización de pipelines por servicio.
Separación por capa técnica	Rompe la cohesión de cada microservicio dispersando sus componentes. Genera mayor acoplamiento entre capas y contradice el principio de independencia de despliegue.

5. Razones de la Decisión

- **Coherencia arquitectónica:** la estructura del repositorio refleja directamente la arquitectura de microservicios adoptada en ADR-001.
- **Independencia de despliegue:** cada servicio en /services/ puede tener su propio Dockerfile, pipeline CI/CD y versionado independiente.
- **Trazabilidad:** los ADRs se ubican en /docs/, garantizando que la documentación arquitectónica esté centralizada y versionada junto al código.
- **Escalabilidad del equipo:** nuevos servicios se añaden creando un nuevo directorio bajo /services/ sin impactar los existentes.
- **Visibilidad global:** el monorepo permite ver el estado de todo el sistema en un solo lugar, facilitando coordinación y revisiones cruzadas.

6. Buenas Prácticas Aplicadas

Práctica	Descripción
README por servicio	Cada directorio en /services/ contiene su propio README.md con instrucciones de instalación, variables de entorno y endpoints expuestos.
Nomenclatura kebab-case	Los directorios usan kebab-case (e.g., clinical-history) para compatibilidad entre sistemas operativos y consistencia.
ADRs en /docs/	Todas las decisiones arquitectónicas se documentan en Markdown dentro de /docs/ con numeración secuencial.
Infrastructure as Code	/infrastructure/ contiene archivos declarativos (docker-compose, k8s manifests) para reproducibilidad de entornos.

Práctica	Descripción
/shared/ para contratos	Contratos de API (OpenAPI/AsyncAPI), tipos compartidos y utilidades comunes residen en /shared/ para evitar duplicación.
.gitignore global	Un .gitignore en la raíz cubre los patrones comunes (node_modules, __pycache__, .env) para todos los servicios.

7. Consecuencias

Positivas

- Estructura clara que cualquier miembro del equipo puede entender de inmediato.
- Pipelines CI/CD configurables por servicio para despliegues independientes.
- La documentación (ADRs) evoluciona junto al código, garantizando trazabilidad histórica.
- Facilita la incorporación de nuevos microservicios sin refactorizar la estructura existente.

Riesgos y Mitigación

Riesgo	Mitigación
Crecimiento del tamaño del repo	Usar Git LFS para artefactos binarios y archivar servicios deprecados en ramas separadas.
Conflictos en /shared/	Establecer pull requests con al menos 2 aprobaciones para cambios en código compartido.
Complejidad del CI/CD monorepo	Usar herramientas como Nx o filtros de path en GitHub Actions para ejecutar pipelines solo cuando cambia un servicio específico.

8. Relaciones con Otros ADRs

ADR-001: Selección de Estilo Arquitectónico (Microservicios) — La estructura del repositorio definida en este ADR es consecuencia directa de la decisión de adoptar microservicios en ADR-001. Cada servicio en /services/ corresponde a uno de los microservicios identificados en el sistema propuesto.

ADR-003: Estrategia de Comunicación REST + OpenAPI — Los contratos de API generados con OpenAPI 3.0 se almacenan en /shared/contracts/, aplicando directamente las convenciones de estructura definidas en este ADR.

ADR 003: Estrategia de Comunicación

Sistema de Gestión Clínica Veterinaria — REST y OpenAPI

Información General

Campo	Valor
ID	ADR-003
Título	Estrategia de Comunicación entre Microservicios
Estado	Aprobado
Fecha	2026
Autoras	Manuela Escobar Hernández Sara Corrales Jaramillo
Proyecto	Sistema de Gestión Clínica Veterinaria
Docente	Joseph Mateus Raigoza Mesa

1. Contexto

VetCare System está compuesto por seis microservicios independientes (ADR-001) que necesitan comunicarse entre sí y con los clientes (app web, app móvil, panel administrativo). La elección del mecanismo de comunicación determina el rendimiento, la mantenibilidad, la interoperabilidad y la facilidad de prueba del sistema.

Se requiere una estrategia de comunicación que sea: uniforme en todos los servicios, comprensible por todos los roles del equipo (desarrolladores, QA, stakeholders), y que facilite la evolución de las APIs sin romper a los consumidores existentes. Adicionalmente, los contratos de comunicación deben ser formales, versionados y validables de forma automatizada.

2. Decisión

Se adopta **REST sobre HTTP** como protocolo único de comunicación para todas las interacciones entre microservicios y con los clientes del sistema. Todos los contratos de comunicación se definen y documentan mediante especificaciones **OpenAPI 3.0**, almacenadas en /shared/contracts/ del monorepo.

La comunicación es completamente síncrona: cada servicio expone endpoints REST bien definidos, y los consumidores realizan llamadas directas y esperan la respuesta. Esto garantiza consistencia inmediata, trazabilidad simple y uniformidad en toda la plataforma.

3. Endpoints REST por Microservicio

Microservicio	Método / Ruta base	Descripción del contrato
user-management	POST /api/v1/auth GET /api/v1/users/{id}	Autenticación con JWT y gestión de usuarios. Contrato OpenAPI define tokens, roles y respuestas de error 401/403.
clinical-history	GET/POST /api/v1/patients/{id}/records	Lectura y escritura del historial clínico. El contrato especifica esquemas de diagnóstico, tratamiento y vacunas.
tracking-reminders	GET /api/v1/reminders POST /api/v1/reminders	Consulta y creación de alertas preventivas. El contrato define filtros por mascota, fecha y tipo de recordatorio.
storage-service	GET/PUT /api/v1/storage/{resource}	CRUD interno de persistencia. Contrato define esquemas de datos, códigos de confirmación y manejo de errores de escritura.
query-visualization	GET /api/v1/dashboard/{petId}	Consultas agregadas para visualización. El contrato especifica las estructuras de respuesta y parámetros de paginación.
notification-service	POST /api/v1/notifications/send	Envío de alertas multicanal. El contrato define canales disponibles (email, push), plantillas y confirmación de entrega.

4. Contratos de Comunicación con OpenAPI 3.0

Cada microservicio define su contrato mediante un archivo `openapi.yaml` almacenado en `/shared/contracts/<nombre-servicio>/. El contrato es la fuente de verdad: la implementación debe ajustarse al contrato, no al revés (Contract-First Development).`

- **Documentación automática:** generación de portales interactivos (Swagger UI, Redoc) directamente desde el contrato.
- **Validación automatizada:** Spectral (linter de OpenAPI) se integra en el pipeline CI/CD y rechaza contratos que incumplan las convenciones del equipo.
- **Generación de código cliente:** los contratos permiten generar SDKs tipados para consumidores internos y externos (openapi-generator).
- **Versionado explícito:** todas las rutas incluyen versión (`/api/v1/...`) para permitir evolución sin romper consumidores existentes.

Ejemplo de contrato OpenAPI — clinical-history

```
openapi: 3.0.3
info:
  title: Clinical History Service API
  version: 1.0.0
  description: Gestión del historial clínico de mascotas – VetCare System
servers:
  - url: https://api.vetcare.co/clinical-history/v1
paths:
  /patients/{petId}/records:
    get:
      summary: Obtener historial clínico de una
```

```

mascota      security:      - bearerAuth: []      parameters:      - name: petId      in:
path          required: true      schema: { type: string, format: uuid }      responses:
'200':          description: Historial clínico retornado exitosamente      content:
application/json:      schema:      $ref: '#/components/schemas/ClinicalRecord'
'401': { description: Token de autenticación inválido o ausente }      '403': { description: Sin
permisos suficientes sobre esta mascota }      '404': { description: Mascota no encontrada }      post:
summary: Registrar nueva consulta médica      requestBody:      required: true      content:
application/json:      schema:      $ref: '#/components/schemas/NewConsultation'
responses:      '201': { description: Consulta registrada exitosamente }      '400': { description:
Datos de consulta inválidos }

```

5. Alternativas Descartadas

Alternativa	Razón del descarte
GraphQL	Ofrece flexibilidad en las consultas, pero introduce complejidad innecesaria para las operaciones CRUD del sistema. El esquema GraphQL no es directamente compatible con OpenAPI, lo que fragmentaría la estrategia de contratos. La curva de aprendizaje es mayor y no hay beneficio claro que justifique el overhead para este caso de uso.
gRPC / Protocol Buffers	Excelente rendimiento para comunicación interna de alta frecuencia, pero su soporte nativo en navegadores es limitado (requiere grpc-web como capa adicional). Los contratos .proto son menos legibles que OpenAPI para equipos multidisciplinares, dificultando la colaboración con QA y stakeholders no técnicos.
SOAP / WSDL	Protocolo maduro con contratos formales, pero excesivamente verboso por el uso de XML. El overhead de procesamiento y la complejidad del estándar son desproporcionados para una arquitectura de microservicios moderna. El ecosistema REST + OpenAPI ofrece las mismas garantías de contrato con mucho menor fricción.
Mensajería asíncrona (RabbitMQ / Kafka)	Si bien la mensajería es adecuada para ciertos patrones, adoptarla como estrategia de comunicación introduce consistencia eventual, dificulta el rastreo de errores y complica los flujos que requieren confirmación inmediata. Para VetCare, la uniformidad y simplicidad de REST en todos los servicios supera los beneficios de la asincronía parcial.
WebSockets	Apropiado para actualizaciones en tiempo real, pero introduce estado persistente en el servidor, complicando el escalado horizontal. Ningún caso de uso de VetCare requiere comunicación bidireccional continua que no pueda resolverse con REST polling o notificaciones push estándar.

6. Razones de la Decisión

- **Uniformidad total:** REST se aplica a los seis microservicios sin excepciones, simplificando la operación, el monitoreo y el onboarding de nuevos desarrolladores.
- **Universalidad:** REST sobre HTTP es compatible con todos los clientes (web, móvil, terceros) sin requerir librerías especializadas ni capas de traducción.

- **Contratos formales con OpenAPI:** la especificación OpenAPI 3.0 es el estándar de la industria para documentar, validar y generar código a partir de APIs REST.
- **Legibilidad:** los archivos openapi.yaml son comprensibles por desarrolladores, QA y stakeholders sin necesidad de conocer el código fuente.
- **Madurez del ecosistema:** amplio soporte de herramientas (Swagger UI, Postman, Stoplight, Spectral, openapi-generator) que cubren todo el ciclo de vida de la API.
- **Alineación con ADR-002:** los contratos OpenAPI se almacenan en /shared/contracts/ del monorepo, siguiendo exactamente la estructura definida en ADR-002.
- **Trazabilidad:** las llamadas REST síncronas son fácilmente observables con herramientas estándar (logs HTTP, Jaeger, Prometheus).

7. Buenas Prácticas Aplicadas

Práctica	Descripción
Contract-First Development	El archivo openapi.yaml se escribe antes de implementar los endpoints. El código se genera o valida contra el contrato — nunca al revés.
Versionado semántico de APIs	Las rutas incluyen versión explícita (/api/v1/, /api/v2/) y los contratos usan SemVer en el campo info.version.
Validación en CI/CD	Spectral valida automáticamente los contratos en cada pull request. Los contratos que incumplan las reglas del equipo no se fusionan.
Timeouts explícitos	Todas las llamadas REST configuran timeouts máximos (5 s) para evitar cascadas de fallos entre servicios.
API Gateway centralizado	Kong o AWS API Gateway actúa como punto de entrada único, gestionando autenticación JWT, rate limiting y logging de forma transversal.
Códigos HTTP semánticos	Se utilizan consistentemente los códigos 200, 201, 400, 401, 403, 404 y 500 en todos los servicios, siguiendo las convenciones definidas en el contrato.
Documentación viva	Swagger UI se despliega automáticamente desde el contrato OpenAPI, garantizando que la documentación siempre refleje la implementación real.

8. Consecuencias

Positivas

- **Interoperabilidad garantizada:** cualquier cliente que respete el contrato OpenAPI puede consumir los servicios sin coordinación adicional.
- **Onboarding acelerado:** nuevos integrantes comprenden todos los endpoints del sistema consultando los archivos openapi.yaml, sin necesidad de leer el código.

- **Pruebas de contrato automatizadas:** herramientas como Pact o Dredd detectan breaking changes antes de llegar a producción.
- **Consistencia inmediata:** al ser comunicación síncrona en todos los servicios, el estado del sistema es predecible y fácil de razonar.
- **Observabilidad uniforme:** todas las interacciones son trazables con las mismas herramientas HTTP estándar.

Riesgos y Mitigación

Riesgo	Mitigación
Desincronización contrato-implementación	CI/CD valida automáticamente con Spectral y pruebas de contrato antes de mergear. El contrato openapi.yaml es la fuente de verdad.
Latencia en cadenas de llamadas REST	Diseñar servicios cohesivos que eviten cadenas de más de 2 saltos síncronos. Usar caching en endpoints de consulta frecuente.
Breaking changes en contratos	Mantener versiones anteriores activas durante un período de deprecación definido. Comunicar cambios con al menos 2 sprints de anticipación a los consumidores.
Carga en picos de tráfico	El API Gateway aplica rate limiting por servicio. Kubernetes escala horizontalmente los microservicios bajo alta demanda, manteniendo la latencia dentro del SLA.

9. Relaciones con Otros ADRs

- **ADR-001:** Los seis microservicios identificados en ADR-001 son los productores de los contratos REST definidos en este ADR. Cada servicio expone su propia especificación OpenAPI.
- **ADR-002:** Los contratos OpenAPI (.yaml) se ubican en /shared/contracts/<nombre-servicio>/ siguiendo exactamente la estructura de monorepo definida en ADR-002.

4. Repositorio de Código Fuente

El código fuente del sistema VetCare (frontend y backend) se encuentra disponible en el siguiente repositorio público de GitHub, organizado como monorepo según se documenta en el ADR-002:

Repositorio principal:

[https://github.com/Manuela582/VetCareSystem Code](https://github.com/Manuela582/VetCareSystem)

Propietaria del repositorio: Manuela Escobar Hernández

Acceso: Público (visibilidad abierta para evaluación académica)

Descripción: Plataforma de gestión clínica veterinaria basada en microservicios.

4.1 Estructura del Repositorio

Siguiendo la estructura de monorepo definida en el ADR-002, el repositorio contiene los siguientes directorios principales (cada microservicio se desarrolla y despliega de forma independiente):

- `/services/user-management/` — Autenticación JWT, autorización por roles (RBAC) y gestión de usuarios.
- `/services/clinical-history/` — Registro, consulta y modificación del historial clínico de mascotas.
- `/services/tracking-reminders/` — Gestión de alertas y recordatorios preventivos (vacunas, controles).
- `/services/storage-service/` — Persistencia centralizada con respaldo y mecanismos de failover.
- `/services/query-visualization/` — Consultas agregadas y visualización del historial para usuarios finales.
- `/services/notification-service/` — Envío multicanal de alertas (email, push, SMS).
- `/shared/contracts/` — Contratos OpenAPI 3.0 (fuente de verdad de las APIs).
- `/infrastructure/` — Manifiestos Docker, Kubernetes y configuración de CI/CD.
- `/docs/` — ADRs y documentación arquitectónica.

4.2 Tecnologías Utilizadas

La solución tecnológica refleja directamente la arquitectura de microservicios diseñada:

- **Frontend Web:** React + JavaScript (interfaz de gestión clínica).
- **Backend (microservicios):** Node.js + Express para servicios web REST.
- **Comunicación:** REST sobre HTTPS, formato JSON, autenticación JWT (ADR-003).

- **Contratos:** OpenAPI 3.0 documentados en /shared/contracts/ con generación automática de Swagger UI.
- **Base de datos:** PostgreSQL (servicios transaccionales) y Redis (cache de consultas).
- **Contenerización:** Docker por servicio (un Dockerfile por microservicio).
- **Orquestación:** Kubernetes para escalado horizontal y alta disponibilidad.
- **CI/CD:** GitHub Actions con pipelines independientes por servicio y validación de contratos vía Spectral.
- **Observabilidad:** Prometheus + Grafana (métricas), Jaeger (trazabilidad distribuida) y logs estructurados centralizados.

5. Diagrama de Arquitectura

El siguiente diagrama presenta la arquitectura general del sistema VetCare, mostrando explícitamente las seis capas que componen la solución y reflejando todas las decisiones documentadas en los ADRs 001, 002 y 003. El diagrama integra: la jerarquía de usuarios, los clientes (frontend web y app móvil), el API Gateway centralizado (punto de entrada único), los seis microservicios independientes con sus endpoints REST principales, la persistencia distribuida (una base de datos por servicio), la infraestructura de despliegue (Docker + Kubernetes) y las herramientas de observabilidad.

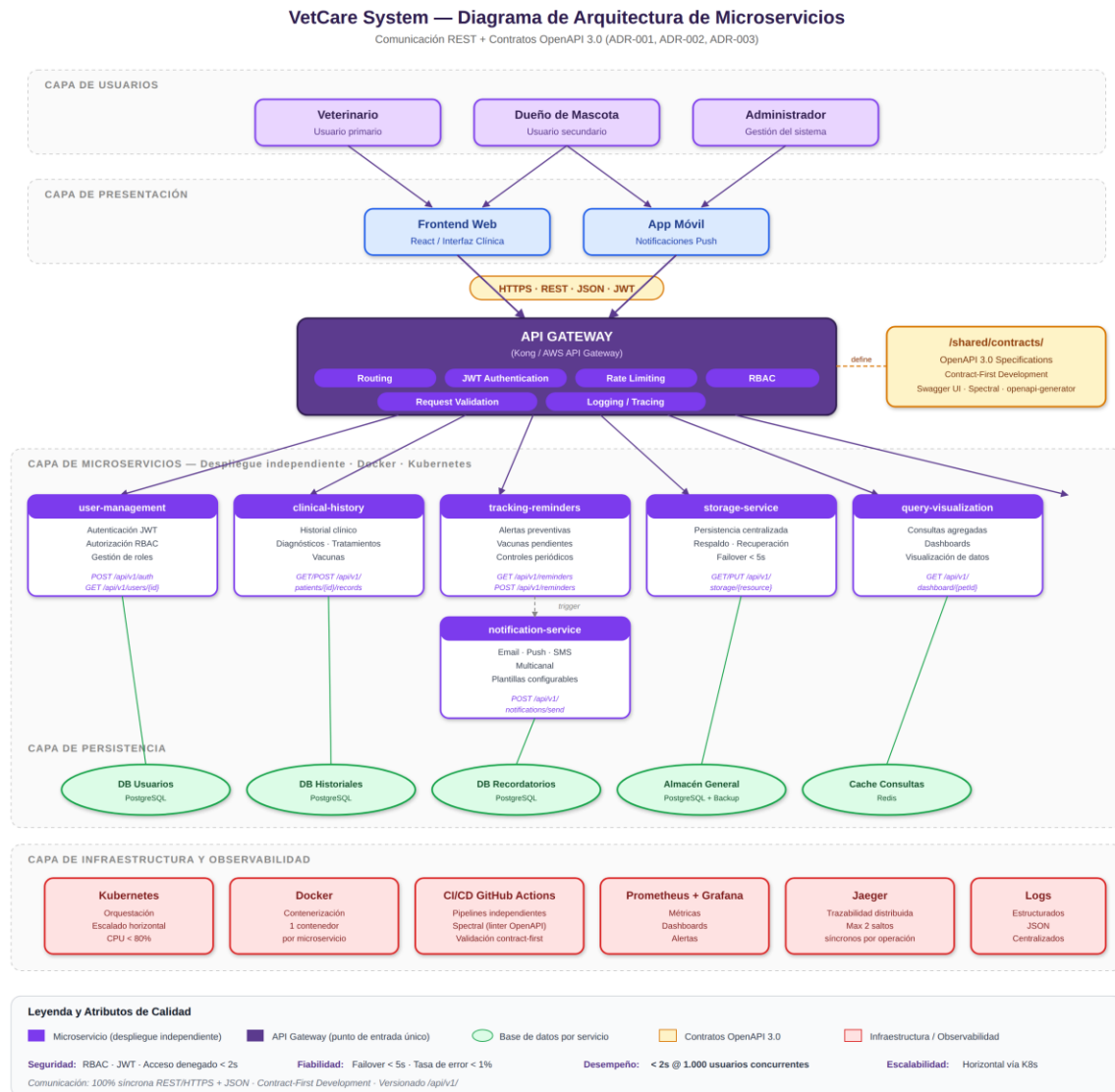


Figura 1. Diagrama de arquitectura del sistema VetCare — arquitectura de microservicios con comunicación REST y contratos OpenAPI 3.0.

5.1 Lectura del Diagrama (de arriba hacia abajo)

- **Capa de Usuarios:** tres tipos de actores (veterinario, dueño de mascota, administrador) cada uno con sus propios permisos según el RBAC del user-management.
- **Capa de Presentación:** frontend web (React) y app móvil consumen el sistema vía HTTPS/REST/JSON con autenticación JWT.

- **API Gateway:** punto único de entrada que centraliza routing, autenticación JWT, validación de requests, rate limiting, control de acceso (RBAC), logging y trazabilidad. Implementado con Kong o AWS API Gateway según se establece en el ADR-003.
- **Contratos OpenAPI 3.0:** almacenados en /shared/contracts/ del monorepo, son la fuente de verdad de cada API (Contract-First Development) y alimentan al Gateway.
- **Capa de Microservicios:** los seis servicios independientes definidos en el ADR-001 (user-management, clinical-history, tracking-reminders, storage-service, query-visualization y notification-service). Cada uno expone su propio endpoint REST y se despliega de forma autónoma.
- **Capa de Persistencia:** cada microservicio cuenta con su propia base de datos (principio de "database per service"), lo que garantiza el aislamiento y la independencia de despliegue exigida por la arquitectura.
- **Capa de Infraestructura y Observabilidad:** Kubernetes orquesta el escalado horizontal manteniendo CPU < 80% en picos; Docker contenedoriza cada servicio; GitHub Actions automatiza CI/CD; Prometheus, Grafana y Jaeger proveen métricas, dashboards y trazas distribuidas.

5.2 Trazabilidad con los Atributos de Calidad

El diagrama refleja explícitamente cómo cada atributo de calidad se materializa en la arquitectura:

- **Seguridad:** JWT y RBAC en el API Gateway (acceso denegado < 2 s, Escenario 1) y validación adicional por servicio (Escenario 2).
- **Fiabilidad:** storage-service con failover < 5 s (Escenario 3); cada microservicio aislado evita fallos en cascada (Escenario 4).
- **Eficiencia de Desempeño:** query-visualization con cache Redis y respuesta < 2 s para 1.000 usuarios concurrentes (Escenarios 5 y 6).
- **Escalabilidad:** Kubernetes escala horizontalmente solo el servicio afectado sin tocar los demás (Escenarios 7 y 8).

6. Solución Tecnológica

La solución tecnológica desarrollada refleja fielmente la arquitectura diseñada y documentada en los ADRs. A continuación se describe brevemente el alcance funcional implementado y disponible en el repositorio:

6.1 Funcionalidades Implementadas

- **Gestión de usuarios y autenticación:** registro, login con generación de JWT, control de acceso por roles (veterinario, dueño de mascota, administrador).
- **Historial clínico:** creación, consulta y modificación de consultas médicas, diagnósticos, tratamientos y registros de vacunación por mascota.
- **Recordatorios y seguimiento:** generación automática de alertas para vacunas pendientes y controles periódicos.
- **Visualización:** dashboard con el historial completo de cada mascota, accesible tanto para veterinarios como para dueños (con permisos diferenciados).
- **Notificaciones:** envío de recordatorios por email y notificaciones push a la app móvil.
- **Documentación viva:** Swagger UI desplegada automáticamente desde los contratos OpenAPI de cada servicio.

6.2 Cómo Ejecutar la Solución

El repositorio incluye instrucciones detalladas en el archivo README.md principal y en el README específico de cada microservicio. A nivel general, los pasos son:

1. Clonar el repositorio: `git clone https://github.com/Manuela582/VetCareSystem_Code.git`
2. Levantar el entorno completo con Docker Compose: `docker-compose up`
3. Acceder al frontend en `http://localhost:3000`
4. Consultar la documentación interactiva de las APIs en `http://localhost:<puerto>/api/docs`

7. Cierre del Documento

Este documento completa los entregables solicitados para el proyecto VetCare System: descripción del problema con cifras, abstract técnico en inglés, cuatro atributos de calidad con ocho escenarios concretos y medibles, tres Architecture Decision Records (ADR-001 selección del estilo arquitectónico, ADR-002 estructura del repositorio y ADR-003 estrategia de comunicación), diagrama de arquitectura y enlace al repositorio con la solución tecnológica funcional.

La arquitectura propuesta responde a una necesidad real del mercado colombiano (≈63 % de hogares con mascotas según el DANE) y materializa los principios de microservicios, monorepo

y comunicación REST con contratos OpenAPI 3.0 para garantizar seguridad, fiabilidad, eficiencia de desempeño y escalabilidad.

— *Fin del documento* —

Arquitectura de Software 2026

Manuela Escobar Hernández · Sara Corrales Jaramillo

Exposición + Demo: 22 / 05 / 2026